



Microsoft

TECHNICAL DESIGN & DELIVERY DOSSIER

Silent Auth Continuity & Dynamic Role-Aware Prompts

Two production-grade enhancements to the **Agent Sidecar** — the Copilot Studio agent embedded in Dynamics 365 model-driven forms. One keeps the conversation alive for the length of a real work session; the other makes the agent context-aware on every form.

Status **Live in Contoso-Dev**

PR #3 · Open · Mergeable

Schema footprint 1 column · no new table

Identity Delegated (per-user)

What we built, and why it matters

The Agent Sidecar surfaces a Copilot Studio agent in a side pane on Dynamics 365 forms, authenticated as the *signed-in user* (delegated identity) so its answers respect that user's permissions. Two gaps limited it in the field. We closed both — with a single additive configuration column (no new table), no new interactive sign-in surface, and the delegated-identity model fully preserved.

Feature 1

Silent auth continuity

Problem: the delegated access token was captured *once* at pane load and never refreshed. Around the 60–90 minute mark the token expired, the agent started returning `401` s mid-session, and the user was forced to re-authenticate.

Solution: a proactive silent-refresh loop that renews the token ~5 minutes before expiry over a CORS call, hot-swaps it into the live conversation *without losing the transcript*, retries transient failures, and falls back to a one-click "Reconnect" chip only when a real interactive re-auth is unavoidable.

Feature 2

Dynamic role-aware prompts

Problem: the pane offered no starting points, or generic ones. Users stared at an empty box and didn't know what the agent could do for *this* record.

Solution: form-specific suggested-prompt chips, keyed by the table's logical name and filtered by the user's security roles. Clicking a chip sends its text *with full record/form context attached*. Prompts ship as a bundled catalog and can be overridden per-table from an in-app admin editor — persisted as JSON on a dedicated `maftagsc_prompts` column added to the existing config table (one additive column, no new table).

1

Additive Dataverse column (maftagsc_prompts) on the existing config table — no new table; the runtime degrades gracefully without it.

~5 min

Refresh skew — a fresh token is in hand before the old one could ever be presented.

1 click

Worst-case recovery — the "Reconnect" chip resumes the *same* conversation, usually with zero prompts.

What it cost to build — AI vs. by hand

This entire delivery — two production features, automated tests, a full documentation set, live deployment, and this interactive presentation — was built with an AI coding agent in about a day and a half, on roughly **10 hours** of human direction and review. Here is the metered AI cost *plus* that human time, compared line-for-line against what the same scope would cost a developer to hand-build at **\$150/hour**.

AI COMPUTE — METERED

\$326

826 Copilot credits · 5h 9m active

YOUR DIRECTION TIME

~10 hrs

≈\$1,500 @ \$150/hr · measured from the session

ALL-IN AI-ASSISTED COST

~\$1,826

\$326 compute + ~\$1,500 your time

SAME SCOPE, BY HAND

\$18,000

≈120 dev-hours @ \$150/hr · ~3 weeks

Net result: **≈10× lower cost** all-in (≈55× on compute alone), about **\$16,200 saved**, delivered in **~1.5 days** instead of ~3 weeks.

Cost, to scale

Every hour of your direction is counted at \$150/hr. The AI-assisted bars are still barely there.

By hand		\$18,000
AI, all-in		\$1,826
AI compute		\$326

Your **~10 hours** of direction produced roughly **120 hours** of engineering output — about a **12× multiplier** on your time.

Two honest ways to read the AI cost

INFERENCE LIST-PRICE

\$326.12

What the raw compute would cost at public per-token list rates (cache-aware, Claude Opus 4.8). The conservative, apples-to-apples “cost to produce.”

COPILOT PREMIUM REQUESTS

826 credits

≈ **\$33** at the \$0.04/request overage rate — and comfortably inside a single Copilot seat’s monthly premium-request allowance.

Either lens tells the same story: the marginal cost of shipping this work with AI is a rounding error against a developer’s time.

How the \$18,000 was estimated — bottom-up, by workstream

A senior developer's effort to hand-build the **same verified scope** — 43 files, +4,570 lines of code, 8 automated tests, 10 documents, a deployed interactive deck — including research, debugging, testing, and review.

WORKSTREAM	WHAT IT COVERS	HOURS	@ \$150/HR
Discovery & root-cause	Diagnose mid-session 401s; MSAL token lifecycle; Dynamics COOP limits; SDK token-setter spike	10	\$1,500
Silent-auth implementation	Refresh scheduler, silent-refresh loop + retry, login-hint plumbing across 3 files, connection rebuild preserving the chat, reconnect UX	26	\$3,900
Silent-auth tests & verification	Unit tests + 4-layer manual test recipe	10	\$1,500
Dynamic-prompts runtime	Config model, bundled catalog as single source of truth, merge logic, role filtering, chip bar, serialization + validation	22	\$3,300
Prompts automated tests	Config validation, role-filter, catalog merge	6	\$900
Prompt authoring	6 role-tuned prompts × 2 incident tables, deployed live	3	\$450
Documentation set	Design doc, ADR, deployment runbook + appendix, 2 handoff specs, setup guides (10 docs)	13	\$1,950
This presentation	955-line self-contained page, 4 architecture diagrams, 2 interactive demos, responsive + accessible + themed, published to Pages	22	\$3,300
Git / PR / deploy	Fork & branch, 18 commits, PR packaging, deploy to Dev, live verification	8	\$1,200
Total	Bottom-up midpoint (range 97–146 h · \$14,550–\$21,900)	≈120	≈\$18,000

By the numbers

85.3M tokens processed

99.998% cache / context reuse

1.36M tokens generated

5h 9m active compute

826 premium-request credits

~10h your direction

Of 85.3M prompt tokens, only **~1,360 were fresh input** — the rest was cached context replayed at a fraction of the price. That prompt-cache efficiency is the difference between a \$326 build and a naïve “tokens × rate” estimate several times higher.

Methodology & honesty notes.

- **AI cost is metered, not modelled** — taken from GitHub Copilot CLI session telemetry for this workspace and priced cache-aware at public list rates. It is an estimate, not an invoice.
- **The developer figure is a bottom-up estimate** for the same delivered scope at the stated \$150/hr. Actuals vary with a developer’s seniority and familiarity with MSAL, the Copilot Studio SDK, and Dynamics.
- **Your direction time is counted — and measured, not guessed.** The ~10 hours of hands-on direction and review comes from the session’s own timestamps (9.4–11.5 h of engaged activity across ~12 work blocks, 58 turns), and is included in the all-in \$1,826. The AI’s ~5 h of compute ran inside that window.
- **The AI cost shown is a conservative floor** — an earlier exploration session was not metered by the telemetry, so real all-in AI spend is modestly higher and the multiple even larger.

— INTERACTIVE

See it in action

Two live micro-demos, running entirely in this page — simulated, no backend, timings accelerated. They mirror the real pane behaviour: form-scoped prompt chips that carry record context and respect roles, and a delegated token that renews itself silently, degrading to a single-click reconnect only when interaction is genuinely required.

**Incident Report · INC-0042**

form-scoped suggested prompts

Agent

Manager

Summarize this incident

Recommend next steps

Find similar incidents

Draft a status update

Click a prompt — it sends with record + role context attached. Switch to "Manager" to reveal a role-gated chip.

**Delegated access token**

silent-refresh loop

Active

auto-refreshes ~5 min before expiry (accelerated for demo)

Silent refreshes this session: 0

Simulate interaction-required

⌚ Session expired — Reconnect

The reconnect reuses the login hint and resumes the *same* conversation — usually zero prompts.

Where the sidecar lives

The pane is a web resource (`agentSidePane.js`) hosted in an iframe on the model-driven form. It signs the user in with **MSAL.js** against **Microsoft Entra ID**, then talks to the **Copilot Studio** agent through the `CopilotStudioClient` / DirectLine connection rendered by **Web Chat**. Per-environment behaviour (which agent, which tables, which prompts) is read at runtime from a **Dataverse** configuration record.

MICROSOFT DYNAMICS 365 model-driven · browser



Model-driven form

record context · entityName · roles



Agent side pane · iframe

agentSidePane.js
MSAL.js · token lifecycle
Web Chat · CopilotStudioClient
prompt chips · context envelope

[acquireTokenSilent · CORS](#)



Microsoft Entra ID

delegated token issuer · rotating refresh token

[DirectLine · delegated](#)



Copilot Studio agent

answers as the signed-in user

[read config at runtime](#)



Dataverse — configuration

maftagsc_sidecarconfiguration · bindings + prompts

Topology — the pane authenticates against Entra, connects to the Copilot Studio agent with the delegated token, and reads its per-environment behaviour from a Dataverse record.

Non-negotiable constraints preserved

IDENTITY & SIGN-IN

- **Delegated identity** — the agent answers as the signed-in user; the disabled DirectLine token-broker (service identity) is *not* revived.
- **localStorage sign-in handshake** retained; no `acquireTokenPopup` or redirect-bridge (Dynamics COOP breaks those).
- Refresh uses a **CORS POST** against the rotating refresh token — *not* a hidden iframe, so third-party-cookie policies don't break it.

CODE & DATA

- **One additive column** — authored prompts persist as JSON on a dedicated `maftagsc_prompts` column on the existing config table (no new table); the bundled catalog needs no schema change and the repository tolerates the column's absence.
- Code App rules honoured: `src/generated/**` read-only, three-layer architecture, HashRouter, Fluent UI v9.
- Green baseline stays green: typecheck, unit tests, lint, build, plus the model-driven build + node tests.

Silent auth continuity

A Dynamics user often keeps one record open for hours. The original pane captured the delegated token exactly once, in `renderConversation()` (`new CopilotStudioClient(settings, token)`), and never renewed it. When Entra expired that token the live conversation had no way to present a valid one — every subsequent turn `401`'d, and the only escape was a full re-auth that also wiped the transcript.

ROOT CAUSE

Access tokens are short-lived by design (~60–90 min). A single capture guarantees expiry inside a normal work session. The fix is *not* a longer token — it's a loop that renews ahead of expiry and swaps the fresh token into the running conversation without disturbing it.

The refresh lifecycle

On every successful token acquisition the pane arms a one-shot timer for `expiresOn - 5 min` (clamped to a 30 s floor). When it fires, the pane silently re-acquires the token, ends the old DirectLine connection, and re-renders Web Chat **reusing the existing store** — so the transcript and `conversationId` survive the swap — then schedules the next refresh against the new expiry.

🔄 Re-arms against the new expiry — indefinitely

1

Sign in

token + expiresOn captured

2

T – 5 min

scheduleTokenRefresh timer fires

3

Silent renew

acquireTokenSilent · CORS POST

4

Hot-swap

rebuild connection, keep store

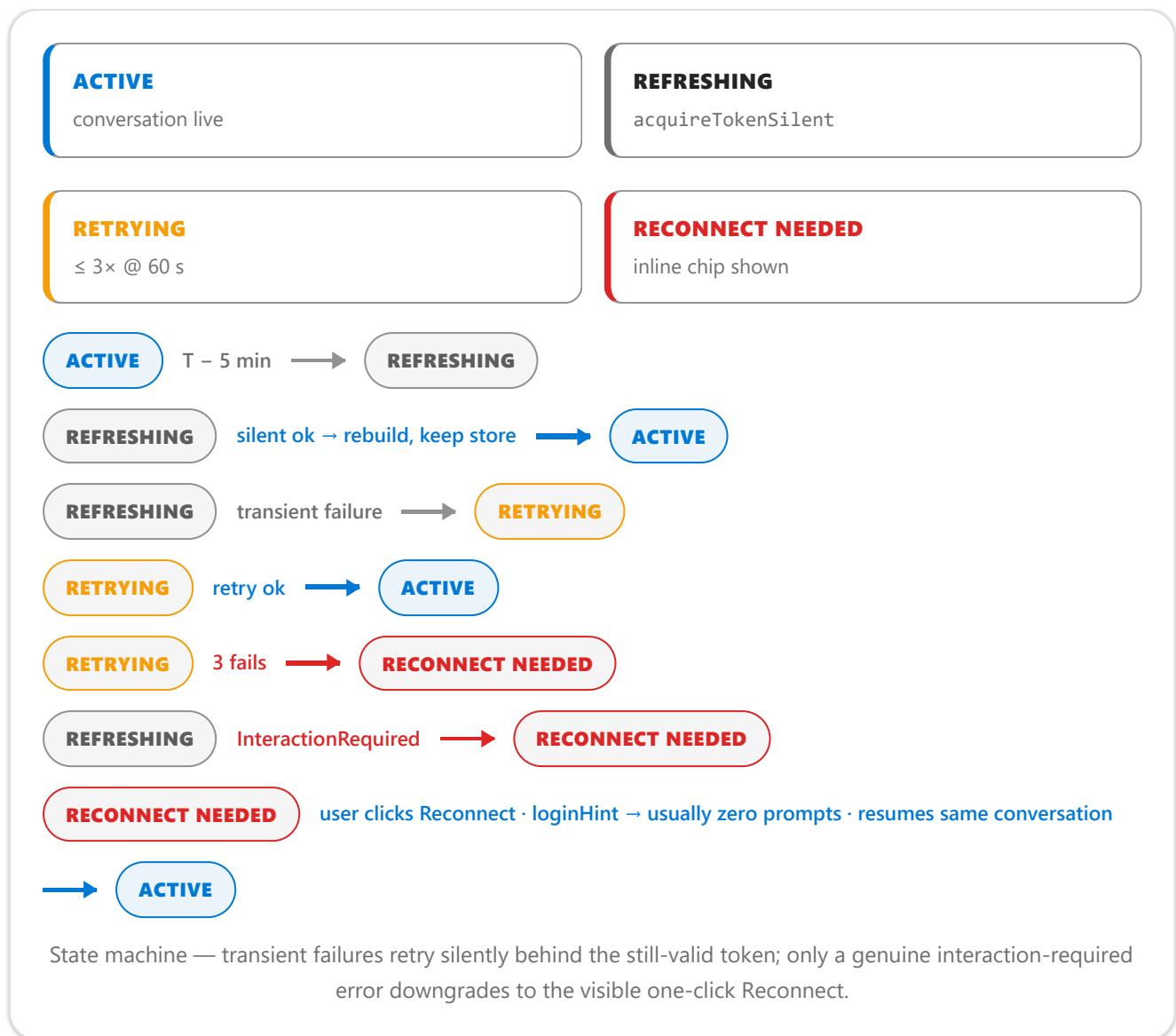


The old token would expire mid-session — but the 5-minute skew keeps a fresh token in hand first, so that point is never reached live.

The 5-minute skew guarantees a valid token is always in hand before the previous one could be presented to the agent.

Acquisition ladder & failure handling

Token acquisition tries the quietest path first and only escalates when it must. Background refreshes distinguish a *transient* failure (retry, the current token is still valid through the skew window) from a genuine *interaction-required* failure (surface a one-click Reconnect).



SSO HINTING KEEPS RE-AUTH INVISIBLE

The launcher captures the Dynamics UPN

(`Xrm.Utility.getGlobalContext().userSettings.userName`) into shared `localStorage`. The pane passes it as MSAL `loginHint` and tries `ssoSilent` before any interactive prompt — so even the "Reconnect" path is typically a single click with no credential entry, courtesy of the existing Entra session.

Key code

Pure timing math — import-free and unit-tested in isolation (`tokenRefresh.ts`):

```
// Refresh 5 min ahead; never schedule tighter than a 30 s floor.
export const TOKEN_REFRESH_SKEW_MS = 5 * 60 * 1000;
export const MIN_TOKEN_REFRESH_DELAY_MS = 30 * 1000;

export function computeRefreshDelayMs(
  expiresOn: Date | null | undefined, now: number,
```

```

    skewMs = TOKEN_REFRESH_SKEW_MS, minDelayMs = MIN_TOKEN_REFRESH_DELAY_MS
  ): number | null {
    if (!(expiresOn instanceof Date)) return null; // unknown lifetime → reactive only
    const expiresAt = expiresOn.getTime();
    if (!Number.isFinite(expiresAt)) return null;
    return Math.max(expiresAt - skewMs - now, minDelayMs);
  }

```

Renew & hot-swap without losing the transcript — the heart of the loop

(agentSidePane.ts):

```

async function refreshActiveToken(configuration) {
  const acquired = await acquireToken(false, configuration, activeLoginHint);
  if (!acquired) { showReconnect(); return; } // interaction genuinely required

  activeToken = acquired.accessToken;
  activeExpiresOn = acquired.expiresOn;
  activeConnection?.end(); // tear down old DirectLine
  resetWebChatHost();
  renderConversation(acquired.accessToken,
    resolveContext(activeContext, configuration),
    configuration, activeStore ?? undefined); // ← reuse store = transcript survives
  scheduleTokenRefresh(configuration); // re-arm against the new expiry
}

```

The `catch` around this call retries a transient failure up to `MAX_SILENT_REFRESH_RETRIES = 3` at 60 s intervals before showing the reconnect chip; the SDK has no token-setter, so rebuilding the client on the same `conversationId` while passing the existing `store` is the sanctioned swap.

Dynamic role-aware prompts

The pane already knew the current table's logical name and the user's roles — it just wasn't using them. We turned that context into a bar of **suggested-prompt chips** above the Web Chat input. Each chip is scoped to a table and, optionally, to specific security roles; clicking one sends its text through the same wrapped `postActivity` the free-text box uses, so the full record and form context rides along automatically.

The prompt pipeline

Prompts come from two sources that merge deterministically. A **bundled catalog** (`promptCatalog.ts`) compiled into the pane guarantees prompts appear in every deployment with zero configuration. An optional **authored JSON blob** on the config record (edited from the admin app) overrides the catalog per-table. `applyPromptCatalog()` only backfills bindings that don't already declare their own prompts — so **authored prompts always win**.



Bundled catalog

promptCatalog.ts · ships in every deployment



Authored JSON

maftagsc_prompts · edited in admin app

authored wins · catalog fills the gaps



applyPromptCatalog()

deterministic merge — never mutates input

filter by role



Role filter

CurrentUserRoles · hides out-of-role chips



Prompt chips

rendered above the Web Chat input

click a chip



postActivity + envelope

same wrapped postActivity as free-text · record / form context attached

Two sources merge (authored beats bundled), roles filter the result, chips render, and a click dispatches the prompt with full context — the same envelope free-text uses.

WHY THE FOOTPRINT IS ONE COLUMN

The bundled catalog means prompts work the moment the pane loads — even against a Dataverse config that has no prompts column yet, because the repository selects that column separately and tolerates its absence. Authored overrides persist as a compact JSON string on a single dedicated memo column — `maftagsc_prompts` — added to the existing config table: **one additive column, no new table**.

Key code

Deterministic merge — pure, never mutates input (`promptCatalog.ts`):

```
// Fill each binding that lacks its own prompts with the catalog entry.
// Binding-level (authored) prompts take precedence; catalog closes the gap.
export function applyPromptCatalog(configuration, catalog = SIDECAR_PROMPT_CATALOG) {
  const entityBindings = {};
  for (const [key, binding] of Object.entries(configuration.entityBindings)) {
    const hasOwnPrompts = Array.isArray(binding.prompts) && binding.prompts.length > 0;
    const catalogPrompts = catalog[key];
    entityBindings[key] = (!hasOwnPrompts && catalogPrompts?.length)
      ? { ...binding, prompts: catalogPrompts } // backfill only
      : binding; // authored wins untouched
  }
  return { ...configuration, entityBindings };
}
```

One serialization boundary — the admin app and runtime agree on shape & caps via

`src/lib/sidecar-prompts.ts` :

```
export const MAX_PROMPTS_PER_TABLE = 6; // per table
const MAX_LABEL_LENGTH = 60; // chip label
const MAX_TEXT_LENGTH = 400; // sent prompt
const LOGICAL_NAME_PATTERN = /^[a-z][a-z0-9_]*$/; // keys = table logical names

// JSON shape persisted on maftagsc_prompts:
// { "contoso_incidentreport": [ { "label": "...", "text": "...", "roles?": ["Manager"] } ] }
```

LIVE IN CONTOSO-DEV TODAY

The configuration record `75b20c59...` ("Incident Reports — Sales Knowledge Agent") carries **6 authored prompts each** for `contoso_incidentreport` and `contoso_incidentprocess` — summarize the incident, recommend next steps, find similar incidents, draft a status update,

explain the process, escalation guidance, and more — all filtered by role and dispatched with record context.

How the customer implements this

Everything is built, tested, and packaged as **PR #3**. Because the customer started from Marty's repository, the path is **review** → **merge** → **build** → **deploy** → **verify** — never re-implementation. The full copy-paste runbook is `docs/customer-implementation-guide.md`; the essentials are below.

Step 1 · Get the code into your repo

Pick the track that matches how you started from Marty's repo. The change lives on branch `feature/auth-refresh-and-dynamic-prompts` (fork `walkthenose75/CustomAgentMDA-1`).

Track A · Forked

SYNC FROM UPSTREAM

After PR #3 merges upstream, sync your fork.

```
git remote add upstream \  
  https://github.com/martycarreras-psnl/CustomAgentMDA.git  
git fetch upstream  
git checkout main  
git merge upstream/main  
git push origin main
```

Track B · Cloned

PULL THE BRANCH NOW

Get it before upstream merges.

```
git remote add sidecar \  
  https://github.com/walkthenose75/CustomAgentMDA-1.git  
git fetch sidecar \  
  feature/auth-refresh-and-dynamic-prompts  
git checkout -b sidecar-auth-prompts \  
  sidecar/feature/auth-refresh-and-dynamic-prompts
```

Track C · Patch

APPLY A REVIEWABLE DIFF

Cleanest audit trail.

```
gh pr diff 3 \  
  --repo martycarreras-psnl/CustomAgentMDA \  
  --patch > sidecar.patch  
git checkout -b sidecar-auth-prompts  
git apply --3way sidecar.patch
```

Step 2 · Review in VS Code

Install the **GitHub Pull Requests** extension, open **PR #3**, and review inline. The change is additive and concentrated in the `copilot/` web resources (auth + prompts), `src/lib/sidecar-prompts.ts`, the admin editor, and one solution entity file (the new column).

THE ONE REAL GATE

PR #3 targets `martycarreras-psnl/CustomAgentMDA`, where the customer currently has **read-only** access. To land it *upstream*, someone with write access (Marty, or an upstream maintainer) clicks merge. To ship without waiting on upstream, merge into the fork's `main` and deploy from there — the same line already proven in Contoso-Dev.

Step 3 · Build & verify locally

```
# this repo uses pnpm (npm works too)  
pnpm install  
pnpm typecheck && pnpm test && pnpm lint && pnpm build  
# compile the deployable web resources (auth + prompts)  
pnpm build:model-driven && pnpm test:model-driven
```

Step 4 · Deploy to Dataverse

#	WHAT	HOW
1	Authenticate	<code>pac auth create --environment https://YOURORG.crm.dynamics.com</code> , then <code>pac org who</code> .
2	The one column	Add <code>maftagsc_prompts</code> (multiline text, 100k) to <code>maftagsc_sidecarconfiguration</code> — via solution import or manually in <code>make.powerapps.com</code> . <i>Needed only for admin-authored prompts.</i>
3	Web resources	Upload the built <code>agentSidePane.html</code> / <code>.js</code> <i>content</i> into your environment's existing side-pane + launcher resources (in a Marty-derived solution: <code>hrAgentSidePane.*</code>) — maker-portal upload, or sync-then- <code>pac solution import --publish-changes</code> . <i>See customer guide §4.0.</i>
4	Admin app optional	<code>pac code push -s "HRAgentSidecar"</code> (the repo's solution unique name) to ship the in-app prompt editor.
5	Publish	Publish all customizations so users get the new resources.

THE ONE SCHEMA CHANGE — STATED PLAINLY

The delivered solution adds exactly **one column**: `maftagsc_prompts` (a JSON memo field) on the *existing* `maftagsc_sidecarconfiguration` table. **No new table.** The runtime is designed to run *without* it — the bundled catalog renders prompts and the repository tolerates the column's absence — so the column is required only when admins author overrides.

Step 5 • Verify

- Prompt chips render on the **Incident Reports** / **Incident Process** forms, filtered by role.
- Clicking a chip sends its text *with record context* attached.
- The pane stays alive past the token lifetime (~60–90 min) with no `401`; worst case, a one-click **Reconnect** chip resumes the *same* conversation.
- Authored prompts (if the column was added) override the bundled catalog on the form.

Recommended GitHub + VS Code workflow

- **Guardrails auto-load.** `.github/instructions/20-agent-sidecar-auth-and-prompts.instructions.md` is path-scoped, so Copilot in VS Code applies the auth/prompt non-negotiables automatically when those files are open.

- **Start from the index.** `AuthPromptPlan.md` in the repo root is the handoff map — status, file anchors, verify commands, and an extension backlog.
- **Extend, don't rebuild.** Issue specs in `docs/handoff/` are ready for the Copilot coding agent (telemetry, configurable skew, admin-authoring polish).

Deep runbook: `docs/deployment-runbook-auth-and-prompts.md` · Full customer guide: `docs/customer-implementation-guide.md` · Copilot Studio agent (Dataverse MCP) instructions for the Incident tables: `docs/copilot-studio-agent-instructions.md`.

Testing & verification

Green baseline

Both feature branches keep the full suite green. Run before merging:

```
npm run typecheck && npm test && npm run lint && npm run build  
npm run build:model-driven && npm run test:model-driven && npm run typecheck:model-driven
```

Silent-auth test recipe (four layers)

1 · UNIT

`npm run test:model-driven` exercises `computeRefreshDelayMs` (skew/floor math) and the `loginHint` / `ssoSilent` plumbing.

2 · OBSERVE THE REAL REFRESH

Open a record form, DevTools → Network, watch the silent `.../oauth2/v2.0/token` POST fire ~5 min before `expiresOn`; the chat never blinks.

3 · FORCE A FAST REFRESH

Temporarily raise `TOKEN_REFRESH_SKEW_MS` above the token lifetime → `computeRefreshDelayMs` clamps to the 30 s floor, so a refresh fires within ~30 s. Confirm the chat survives on the same `conversationId`. **Revert the skew after.**

4 · FORCE RECONNECT

Make `acquireTokenSilent` throw `InteractionRequiredAuthError` (sign out in another tab / clear the MSAL cache entry); the inline "Reconnect" chip must resume the same thread. Transient failures retry 3× at 60 s first.

Prompt smoke test

- Open an `contoso_incidentreport` record → the incident chips render above the input.
- Click one → it dispatches with record context and the agent answers about *that* incident.
- Sign in as a user missing a gated role → role-scoped chips are hidden.
- Author a prompt override on the config record → it replaces the bundled catalog for that table.

07 — STATUS

Deployment & pull request

Pull request

MERGEABLE

no conflicts

no required checks

PR #3 → `martycarreras-psn1/CustomAgentMDA`. Both features, with a review-and-merge description (summary, what-to-review, green-baseline commands, deploy state). Nothing blocks the merge except upstream write access.

[View PR #3 ↗](#)

Live environment

Deployed Contoso-Dev

- Runtime web resources — silent refresh, reconnect chip, prompt chips.
- The portable `maftagsc_prompts` column.
- Admin code app (in-app prompt authoring).
- Authored prompts on record `75b20c59...` for both incident tables.

File & commit map

AREA	FILE	ROLE
Auth	<code>model-driven/.../copilot/tokenRefresh.ts</code>	Pure refresh-delay math (skew/floor); unit-tested.
Auth	<code>model-driven/.../copilot/agentSidePane.ts</code>	Acquisition ladder, refresh scheduler, connection rebuild, reconnect chip.
Auth	<code>model-driven/.../copilot/agentSidePaneLauncher.ts</code>	Captures the Dynamics UPN for <code>loginHint</code> / <code>ssoSilent</code> .
Prompts	<code>model-driven/.../copilot/promptCatalog.ts</code>	Bundled defaults + <code>applyPromptCatalog()</code> merge (authored wins).
Prompts	<code>src/lib/sidecar-prompts.ts</code>	Single serialization boundary — shape, caps, sanitization.
Prompts	<code>model-driven/.../copilot/sidecarConfigurationRepository.ts</code>	Reads authored <code>maftagsc_prompts</code> (tolerant of a missing column).
Docs	<code>AuthPromptPlan.md</code>	Handoff index — status, anchors, verify commands, backlog.
Docs	<code>docs/handoff/issue-*.md</code>	Coding-agent-ready issue specs for the extension backlog.
Deploy	<code>docs/deployment-runbook-auth-and-prompts.md</code>	Turnkey deploy routes, rollback, greenfield appendix.
Deploy	<code>docs/customer-implementation-guide.md</code>	Step-by-step fork → build → deploy → verify for the customer.
Agent	<code>docs/copilot-studio-agent-instructions.md</code>	Copilot Studio agent instructions (Dataverse MCP) for the Incident tables.

SELECTED COMMITS ON THE BRANCH

COMMIT	SUMMARY
c90c1dc	docs(auth): 4-layer silent-refresh test recipe in the handoff
ca08432	docs(handoff): AuthPromptPlan + scoped Copilot instructions + issue specs
bb9fa63	feat: in-app authoring UI for role-aware suggested prompts
b86a2aa	Add suggested prompts for incident tables
7301a5c	fix: make authored-prompt column portable; pane resilient to its absence

Agent Sidecar — Silent Auth Continuity & Dynamic Role-Aware Prompts. Delegated identity · one additive config column · no new table.

Source: [walkthenose75/CustomAgentMDA-1](#) · Pull request: [PR #3 ↗](#) · Self-contained page at [docs/presentation/index.html](#) . Supports light/dark, reduced-motion, and print-to-PDF.